The BIM Intent Compiler

WHITE PAPER

The BIM Intent Compiler: A DSL-Driven DAG Compiler for Spatially-Verified IFC Output from Declarative Building Descriptions

Redhuan D. Oon CTFL

red1org@gmail.com

Subject Matter Expert, ERP Software, Fellow of

Entruss Ventures Sdn Bhd (1186369-A)

No. 15, Jalan 1/3c, Seksyen 1,
43650 Bandar Baru Bangi,
Selangor D. E.

Tel: 03 - 8925 8301

Email: info@entruss.net

Version 1.0 February 13, 2026

Table of Contents

Abstract

Building Information Modelling (BIM) authoring remains a manual, error-prone process dominated by graphical tools that conflate design intent with geometric detail. We present the BIM Intent Compiler, a system that compiles declarative building descriptions written in a domain-specific language (DSL) into spatially-verified Industry Foundation Classes (IFC) output through directed acyclic graph (DAG) expansion. The compiler implements a three-tier architecture separating concerns across a user-facing DSL layer (catalog selection), an expert-maintained metadata layer (assembly recipes and placement rules stored in SQLite), and a developer-maintained resolver layer (Java compilation engine). Building elements are organized in a six-level assembly hierarchy inspired by OSGi component models, where each level is a bill-of-materials (BOM) of the level below. To validate compiler fidelity, we introduce the Rosetta Stone Methodology, an empirical framework adapted from historical linguistics: reference IFC models from real buildings serve as ground truth "stone tablets," and a spatial X-ray algorithm compares compiled output against reference using dimension-signature fingerprinting with asymmetric tolerances. We evaluate the system against three construction traditions — UK residential, US residential, and Malaysian institutional — achieving 62%, 53%, and 9% architectural fidelity respectively across 14 grammar rules discovered from first principles. The methodology is reproducible, measurable, and incremental: each compiler improvement manifests as a quantifiable score change. We identify transferable idioms (ZONE, REFERENCE_SYSTEM, DECOMPOSITION, BOUNDARY) that generalize the building grammar toward infrastructure domains, and discuss the path to 100% spatial convergence. The system demonstrates that declarative compilation of BIM models is feasible, verifiable, and can capture expert construction knowledge in queryable metadata rather than procedural code.

Keywords: BIM, IFC, domain-specific language, compiler design, spatial verification, building automation, directed acyclic graph, assembly hierarchy

---

1. Introduction

1.1 The BIM Authoring Problem

Building Information Modelling has transformed the architecture, engineering, and construction (AEC) industry by enabling digital representation of physical building characteristics [1, 2]. Yet the creation of BIM models remains fundamentally a manual process. Practitioners use graphical authoring tools — Autodesk Revit, Bentley OpenBuildings, Graphisoft ArchiCAD — to place individual building elements one at a time, specifying geometry, properties, and relationships through mouse interactions and dialog boxes.

This approach has three structural deficiencies:

1. **Conflation of intent and detail.** The designer's intent ("a three-bedroom house with an exterior wall on the west face") is inseparable from the geometric realization (wall coordinates, layer thicknesses, opening positions). Changing intent requires re-editing hundreds of geometric parameters.

2. **Non-reproducibility.** Two modellers given the same brief will produce different models. There is no deterministic path from building description to BIM output.

3. **Knowledge burial.** Construction knowledge — that a Malaysian exterior wall is 150mm brick-plaster while a UK equivalent is 290mm cavity brick — lives in the modeller's head or scattered across project templates, not in queryable, version-controlled data.

1.2 The Compilation Approach

We propose treating BIM model creation as a compilation problem. A building description, written in a purpose-designed DSL, serves as source code. A metadata catalog of construction components, assemblies, and placement rules serves as the "standard library." A compiler resolves the declarative intent against the catalog and produces a complete, positioned IFC model as output.

This is analogous to how a programming language compiler transforms high-level source into executable machine code: the programmer expresses *what* the program should do; the compiler determines *how* to realize it in the target representation. The key insight is the same separation of concerns: end users describe intent, experts curate the standard library, and the compiler bridges the gap deterministically.

1.3 Contributions

This paper makes the following contributions:

- A **three-tier architecture** (DSL / Metadata / Resolver) that separates user intent from construction knowledge from compilation logic, enabling non-programmers to create building variants through metadata curation alone.
- A **six-level assembly hierarchy** modelled on OSGi component contracts, where buildings decompose through floors, units, rooms, fixture arrangements, and components via DAG expansion.
- The **Rosetta Stone Methodology**, an empirical validation framework adapted from historical linguistics, using real IFC files as ground truth and a spatial X-ray scoring algorithm as the loss function.
- **14 grammar rules** discovered from first principles across three construction traditions (UK, US, Malaysian), demonstrating that construction knowledge can be formalized as mathematical relations rather than fitted constants.
- An **open-source implementation** in Java 17 with SQLite metadata, evaluated against three reference buildings totalling over 5,000 architectural elements.

1.4 Paper Organization

Section 2 reviews related work in IFC standards, BIM automation, DSL theory, and compiler design. Section 3 presents the Rosetta Stone Methodology. Section 4 describes the system architecture. Section 5 details the software design patterns employed. Section 6 provides implementation details with code extracts. Section 7 presents DSL examples. Section 8 reports empirical results. Section 9 discusses challenges and lessons learned. Section 10 outlines future directions. Section 11 concludes.

---

2. Related Work

2.1 The IFC Standard

The Industry Foundation Classes (IFC), standardized as ISO 16739 [3], define an open data model for describing building and construction data. The IFC4 specification [4] provides over 800 entity types organized in an inheritance hierarchy, covering spatial structure (IfcSite, IfcBuilding, IfcBuildingStorey, IfcSpace), building elements (IfcWall, IfcDoor, IfcWindow, IfcSlab), MEP systems (IfcFlowSegment, IfcFlowTerminal), and structural elements (IfcBeam, IfcColumn). While IFC provides the *vocabulary* for describing buildings, it does not prescribe *how* to compose that vocabulary into valid building models — a gap our compiler addresses.

Venugopal et al. [5] analyzed the semantic richness of IFC and identified that much of the standard's expressive power remains unused by commercial tools, which tend to export a minimal subset. Our system targets a richer IFC output by driving element generation from metadata-defined assembly recipes rather than tool-specific export filters.

2.2 BIM Automation and Rule Checking

Eastman et al. [1] established the foundational vision for BIM as an integrated digital building representation. The second and third editions of the BIM Handbook [2] extended this vision to include automated model checking and multi-discipline coordination. Preidel et al. [6] surveyed BIM-based rule checking approaches, distinguishing between geometric rule checking (clash detection, clearance verification) and semantic rule checking (code compliance, design rule validation). Our system performs both: geometric verification through spatial digest comparison, and semantic validation through the CatalogContract mechanism.

Won and Lee [7] investigated automated BIM model generation from design rules, demonstrating that parametric approaches can reduce modelling effort. However, their approach remained within the graphical authoring paradigm. Pauwels et al. [8] explored Linked Building Data as a semantic web approach to BIM interoperability, emphasizing the importance of formal ontologies. Our metadata catalog serves a similar function to an ontology but is expressed as relational tables rather than RDF triples, prioritizing query performance and practitioner accessibility.

Lee et al. [9] proposed automated rule-based checking of building designs using IFC models, while Solihin and Eastman [10] classified rule checking approaches into four categories. Our compiler's validation chain (CatalogContract, SpatialDigest, WitnessGenerator) implements elements from multiple categories within a unified compilation pipeline.

2.3 Domain-Specific Languages

Fowler [11] provides the comprehensive treatment of DSL design and implementation, distinguishing between internal DSLs (embedded in a host language) and external DSLs (with their own parser). Our BIM DSL is an external DSL with a custom recursive-descent parser, chosen to provide a syntax accessible to construction professionals without programming experience.

Mernik, Heering, and Sloane [12] surveyed DSL development methodology, identifying patterns for when and how to create domain-specific languages. Their analysis of the "notation" pattern — providing domain-appropriate syntax for domain concepts — directly motivates our DSL design: keywords like `BUILDING`, `STOREY`, `BEDROOM`, and `WINDOW` correspond to IFC spatial concepts while abstracting away geometric parameterization.

Van Deursen, Klint, and Visser [13] characterized DSLs as offering increased productivity, maintainability, and domain expert accessibility at the cost of language design effort and potential performance overhead. Our experience confirms these trade-offs: the DSL enabled a progression from 0% to 62% spatial fidelity through metadata-only changes (no DSL modifications), but required substantial upfront investment in parser design and resolver architecture.

2.4 Compiler Design

Aho, Lam, Sethi, and Ullman [14] define the classical compiler pipeline as a sequence of phases: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and code generation. Our BIM compiler follows this structure with domain-specific adaptations:

Classical Phase	BIM Compiler Phase	Implementation
Lexical analysis	DSL tokenization	Recursive-descent parser
Syntax analysis	Building definition tree	`BuildingDefinition` record hierarchy
Semantic analysis	Catalog validation	`CatalogContract.validateCatalog()`
IR generation	Storey compilation	`StoreyCompiler.compileStorey()` pipeline
Optimization	Constraint resolution	`resolveConstraints()`, deduplication
Code generation	IFC output writing	`BuildingWriter.write()` sub-writer delegation

The DAG expansion model — where each assembly level decomposes into a bill-of-materials of the level below — is related to attribute grammar evaluation [14] and tree rewriting systems [15]. However, our DAG is spatial rather than syntactic: each node carries a bounding box, and expansion must respect physical constraints (elements must fit within rooms, rooms within floors).

2.5 OSGi and Component Models

The OSGi Alliance specification [16] defines a component model for Java with explicit module boundaries, versioned interfaces, and declarative service binding. We adopt the OSGi metaphor for our assembly hierarchy: each building assembly exposes a "manifest" declaring what its six faces provide and require, analogous to OSGi's `Export-Package` and `Import-Package` headers. This enables loose coupling between assembly levels — a bathroom assembly can be upgraded from v1.0 to v1.1 without changes to the containing room, provided the face contracts remain compatible.

2.6 The Rosetta Stone and Historical Linguistics

The decipherment of Egyptian hieroglyphics using the Rosetta Stone — a trilingual decree from 196 BCE — provides the methodological analogy for our validation framework. Young's phonetic analysis and Champollion's subsequent full decipherment [17, 18] demonstrated that an unknown writing system can be decoded by systematic comparison with a known parallel text. Robinson [19] traces this methodology through subsequent decipherments (Linear B, Mayan), establishing a pattern: dictionary extraction (what symbols exist), thesaurus construction (which symbols correspond across scripts), and grammar inference (what rules govern symbol combination). Our three-step pipeline — Dictionary, Thesaurus, Grammar — follows this pattern exactly, treating reference IFC files as "known scripts" and compiled output as the "unknown script" to be validated.

2.7 Level of Development Specification

The BIMForum Level of Development (LOD) Specification [20] defines progressive levels of geometric and informational completeness for BIM elements: LOD 100 (conceptual), LOD 200 (approximate geometry), LOD 300 (precise geometry), LOD 350 (precise geometry with connections), and LOD 400 (fabrication-ready). Our component library stores LOD 400 tessellated geometry extracted from reference IFC files, ensuring that compiled output meets the highest geometric fidelity level while the DSL operates at what might be termed "LOD 0" — pure intent with no geometry.

---

3. The Rosetta Stone Methodology

3.1 Motivation

Validating a BIM compiler is harder than validating a programming language compiler. A C compiler can be tested against a conformance suite with defined inputs and outputs. A BIM compiler must produce spatially correct three-dimensional geometry — a domain where visual inspection is non-reproducible and manual comparison is tedious.

We needed a validation methodology that is: - **Reproducible**: the same reference always produces the same ground truth. - **Measurable**: a single comparable number tracks progress. - **Incremental**: each compiler fix shows up as progress. - **Honest**: the compiler must use its own resolution motions, not copy the reference. - **Generalizable**: proven across multiple construction traditions.

3.2 Rosetta Stone Pairs

Each validation pair consists of four artifacts:

1. **Source IFC** — an unmodified IFC file from a real building, serving as ground truth.
2. **Extracted DB** — bounding boxes and metadata extracted from the IFC into a SQLite database.
3. **DSL File** — a building description in our DSL that describes the same building.
4. **Compiled DB** — the compiler's output from the DSL file.

The compiler must be *truthful*: it reads a simple DSL manifest, resolves assemblies from its metadata catalog, places elements according to its grammar rules, and writes positioned IFC elements. The test proves that this process arrives at the same spatial composition as the original IFC — without copying.

We maintain three active pairs spanning three construction traditions:

Pair	Building	Tradition	ARC Elements	Disciplines
1	SampleHouse	UK residential	35	ARC only
2	Duplex	US residential	183	ARC + MEP + STR
3	SJTII Terminal	Malaysian institutional	3,818	8 disciplines

3.3 The Linguist's Method

Our validation pipeline adapts the methodology of historical linguistic decipherment into three steps:

Step 1: Dictionary. Extract every element's spatial anchor points from both the reference and the compiled output. Catalog every fact: wall starts here, door sits there, furniture anchors at this corner. This is implemented by `tools/rosetta_dictionary.py`, which produces an 11-section spatial skeleton for any IFC database.

Step 2: Thesaurus. Map equivalent concepts across construction traditions. A UK 290mm cavity brick exterior wall and a US 417mm frame-with-siding exterior wall are both "EXTERIOR wall" — same part of speech, different dialect. The thesaurus finds common factors and multiples. Critically, rules must be expressed from first principles (wall thickness = sum of layers), not as fitted constants (UK wall = 290mm). First principles generalize; fitted constants overfit.

Step 3: Grammar. Formalize the rules that derive element placement from DSL intent. Each rule must be a mathematical truth common to all stones. For example: `WALL_THICKNESS = SUM(layer_thickness_i)` holds for UK brick-cavity (102+50+102+38 = 290mm), US frame-with-siding (12+140+12+140+100+12 = 417mm), and Malaysian brick-plaster (12+125+12 = 150mm).

3.4 Spatial X-Ray Algorithm

The core scoring mechanism is the *spatial X-ray*: a dimension-signature fingerprint comparison that ignores names, absolute positions, and orientations. It reads spatial "bone structure" — for each element, it creates a signature tuple:

```
signature = (category, L_mm, W_mm, H_mm)
```

where dimensions are sorted in descending order and bucketed to 10mm resolution. The `category` maps IFC classes to broad spatial categories (e.g., both `IfcPlate` from the compiled output and `IfcWall` from the reference map to `WALL`).

Two matching modes are employed:

- **Exact match**: identical signature tuples (same category, all dimensions within the same 10mm bucket).
- **Near match**: same category, thin dimension (smallest) within 5mm absolute tolerance, two larger dimensions within 10% relative tolerance.

Matching is performed greedily: each reference element is matched at most once against the output element pool for its category. The near-match count divided by the reference element count produces the X-ray score.

3.5 Discipline Filtering

Real buildings comprise multiple engineering disciplines: architecture (ARC), mechanical/electrical/plumbing (MEP), structural (STR), fire protection (FP), and others. The spatial checker supports a `--discipline` flag that filters elements by their IFC class before comparison:

```
python3 tools/spatial_checker.py output.db reference.db --discipline ARC
```

This prevents large MEP element counts (e.g., the Duplex's 890 MEP elements) from drowning architectural signals (183 ARC elements) in a blended score. For convergence tracking, we always report the ARC-discipline score as the primary metric.

3.6 Why This Is Sound Science

The Rosetta Stone Methodology satisfies the criteria for empirical software engineering validation:

1. **Construct validity**: the X-ray measures spatial fidelity, which is the core claim of the compiler.
2. **Internal validity**: the compiler cannot copy the reference — it must use its own resolution pipeline.
3. **External validity**: three construction traditions (UK, US, Malaysian) with different wall systems, opening conventions, and furniture standards.
4. **Reliability**: deterministic extraction + deterministic compilation = reproducible scores.

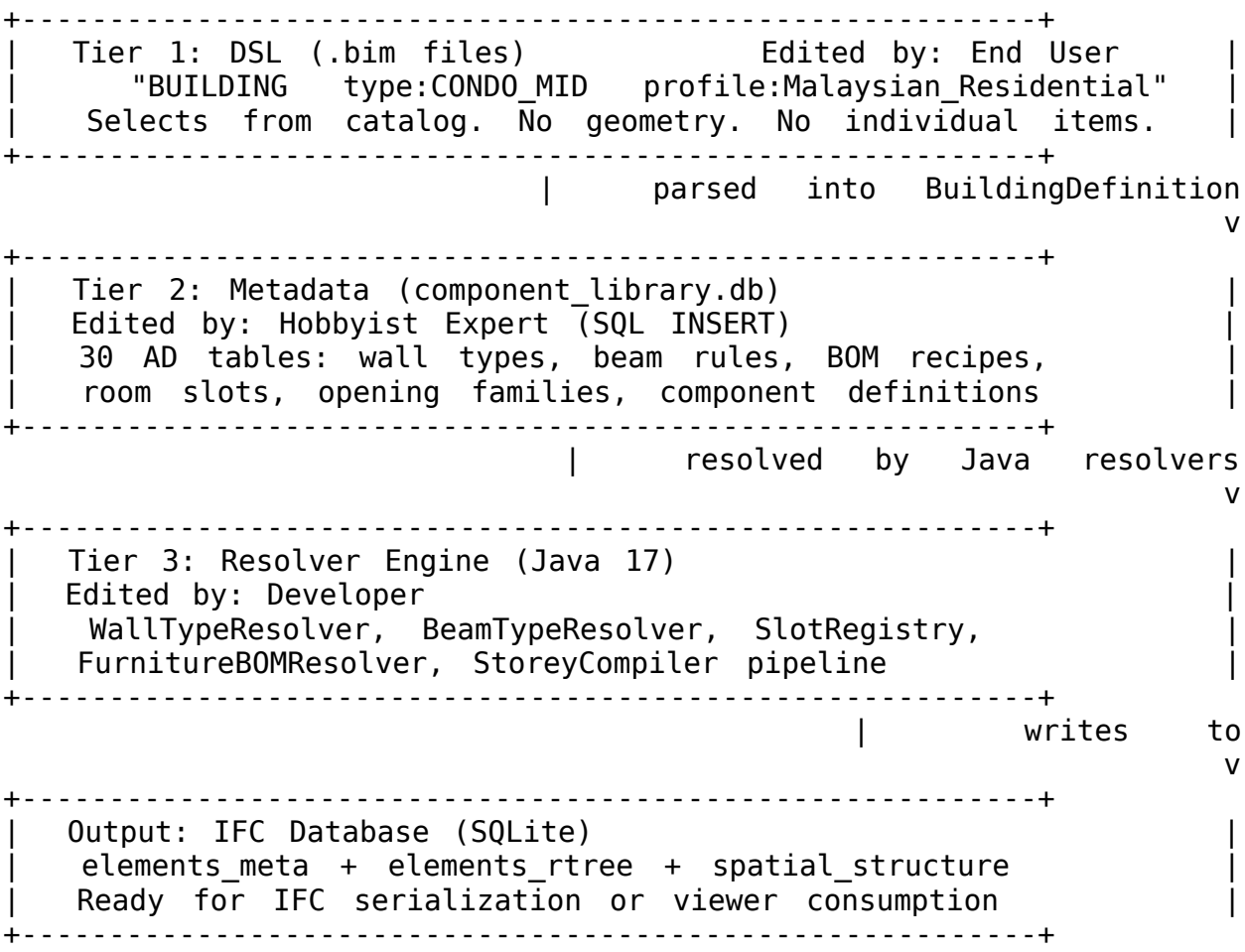
The completion criterion is unambiguous: 100% X-ray fidelity on all three stones. Anything less means the grammar has gaps.

---

4. System Architecture

4.1 Three-Tier Separation

The BIM Intent Compiler separates concerns across three tiers, each maintained by a different stakeholder:



This separation is modelled on the OSGi manifest principle [16]: the DSL is a *manifest* declaring what the user wants from the catalog (like `Require-Bundle`), the metadata is the *catalog* of available components and assemblies (like a bundle repository), and the Java resolvers are the *runtime* that wires everything together (like the OSGi framework).

The practical consequence is that new building variants can be created through SQL INSERT statements in the metadata layer — no Java code changes and no DSL syntax changes required.

4.2 Six-Level Assembly Hierarchy

Building elements are organized in a six-level DAG, where each level is a bill-of-materials (BOM) of the level below:

```
Level   4:   BUILDING                                    "SJTII_Terminal"
                                                                    |
Level  3:  FLOOR                         Ground  Floor,  First  Floor,  ...
                                                                    |
Level  2:  UNIT                              (unit  grouping  —  optional)
                                                                    |
Level 1: ROOM                  LOBBY "main_lobby", OFFICE "admin_1", ...
                                                                    |
Level 0.5: MEP SUB-ASSEMBLY  SPRINKLER_DROP = tee + transition + drop
+                                                                  head
                                                                    |
Level  0:  COMPONENT                    Door_900x2100,  Light_Downlight,
Toilet_WC,                                                          ...
                                                                    |
Level -1: FIXTURE ARRANGEMENT WORKSTATION_STD = desk + chair + visitor
chairs
```

The DSL operates at Level 4 (building) and specifies Level 1 content (rooms). Everything between and below is resolved from metadata. This means the DSL for a 4-storey, 37-room airport terminal is approximately 80 lines — the same order of magnitude as a 3-room house.

4.3 Compilation Pipeline

The compilation pipeline follows a five-stage DAG:

```
Parse  ->  Resolve  ->  Compile  ->  Place  ->  Write
```

Parse: The recursive-descent parser transforms DSL text into a `BuildingDefinition` — a tree of Java records (`StoreyDef`, `RoomDef`, `OpeningDef`, etc.) representing the user's declared intent.

Resolve: The catalog validator (`CatalogContract`) checks that every DSL reference resolves to an existing metadata entry. Wall types, beam rules, room slots, and opening families are loaded from the component library.

Compile: The `StoreyCompiler` executes a 12-step pipeline per storey, transforming abstract room definitions into positioned, dimensioned building elements. This is the core DAG expansion phase.

Place: Specialized placers (furniture, structural, MEP) consume the compiled room envelopes and dispatch placement workers according to the slot registry.

Write: The `BuildingWriter` orchestrates sub-writers (`ElementPersistence`, `StructuralWriter`, `StairWriter`, `OpeningWriter`, `MEPWriter`) to serialize positioned elements into the output database.

4.4 Output Schema

The compiled output is stored in a SQLite database with three core tables:

- **`elements_meta`**: Element identity and classification (IFC class, name, storey, discipline, geometry hash).
- **`elements_rtree`**: Spatial index using SQLite's R-tree extension, storing axis-aligned bounding boxes (minX, maxX, minY, maxY, minZ, maxZ).
- **`spatial_structure`**: The IFC spatial hierarchy (Site > Building > Storey > Space).

This schema enables efficient spatial queries (e.g., "find all elements within 1m of this point") while maintaining IFC semantic structure. The R-tree index supports the spatial X-ray comparison algorithm directly.

---

5. Software Design Patterns

5.1 Lazy Singleton Resolver

Every metadata resolver (wall types, beam types, column types, slot registry) follows the same pattern: a lazy-loaded singleton that reads all relevant metadata from the component library database on first access, then answers queries from an in-memory data structure.

```
SlotRegistry.getInstance()                  // returns cached singleton
    .getSlotsForType("BEDROOM")    // answers from HashMap
```

This pattern provides: - **Single database connection**: metadata is loaded once, not per-query. - **Graceful degradation**: if a metadata table does not exist, the resolver returns null and the caller uses a hardcoded fallback. - **Thread safety**: `ensureLoaded()` is synchronized; subsequent reads are lock-free. - **Uniform API**: all resolvers follow the same `getInstance()` / `ensureLoaded()` contract.

5.2 Two-Pass Profile-Aware Resolution

Many metadata lookups are profile-aware: a Malaysian institutional building uses different wall types, beam sizes, and furniture sets than a UK residential building. All resolvers implement a two-pass resolution strategy:

1. **Pass 1**: Search for rules matching the specific profile (e.g., `Malaysian_Institutional`).
2. **Pass 2**: Fall back to generic rules (profile = NULL).

This means: - Adding a new profile requires only SQL INSERTs — no Java changes. - Existing buildings without a profile continue to work via the generic fallback. - Profile-specific rules automatically override generic rules for the same context.

5.3 BOM Assembly and Slot Dispatch

Room contents are determined by a two-level dispatch:

1. **Slot Registry**: The `ad_room_slot` table maps room types to assembly slots (e.g., `BEDROOM` has a `FURNITURE` slot pointing to `BED_SET`, and a `DINING` slot may or may not be present).

2. **BOM Resolver**: Each assembly (e.g., `BED_SET`) is expanded into its child components via the `ad_bom_child` table (bed, nightstand, wardrobe), with placement parameters from `ad_bom_child_param` (spatial offsets, z-rules, wall-face anchoring).

This separation means that changing what goes in a bedroom (e.g., adding a desk) requires only a SQL INSERT into `ad_bom_child` — no changes to the slot registry, the DSL, or the Java code.

5.4 Sub-Writer Delegation

The `BuildingWriter` orchestrates output generation by delegating to specialized sub-writers:

```
BuildingWriter.write()
      ├── ElementPersistence     (core element writing, rtree, meta)
            ├──   StructuralWriter                  (beams,   columns)
      ├── StairWriter              (stair flights, landings, railings)
      ├── OpeningWriter             (doors, windows — resolves library
geometry)
      └── MEPWriter                (sprinklers, lights, plumbing, HVAC,
electrical)
```

Each sub-writer receives an `ElementPersistence` instance and a database `Connection`, following a uniform contract. This decomposition prevents a monolithic writer class and enables independent evolution of each element category.

5.5 Contract Enforcement

Two contract mechanisms ensure system integrity:

CatalogContract: Validates at parse-time that every DSL reference resolves to an existing metadata entry. A building template, unit type, floor BOM, room type, or opening family that does not exist in the catalog produces a `CatalogViolation`. This catches configuration errors before compilation begins.

SpatialDigest: Computes a SHA-256 hash of all element bounding boxes at 1mm precision after compilation. A change in any element's position or dimensions changes the digest. This serves as a determinism check — the same input must always produce the same digest — and as a regression test.

5.6 BundleWorker Interface

The `BundleWorker` interface defines the contract for placement workers using an OSGi-inspired "construction site" metaphor:

1. Worker ARRIVES with a theme ID (e.g., `BED_SET`, `KITCHEN_COUNTER_SET`).
2. Worker READS the room envelope (bounds, faces, reserved zones).
3. Worker PLACES elements by face alignment and dimension properties.
4. Worker RESERVES its clearance envelope for the next worker.
5. Worker LEAVES — no coupling to other workers.

Workers are dispatched by the slot registry in priority order. Each worker sees only the room envelope and prior workers' reservations, never the workers themselves. This enables independent development and testing of placement strategies.

---

6. Implementation

6.1 SlotRegistry: Two-Pass Profile-Aware Dispatch

The `SlotRegistry` demonstrates the lazy singleton + two-pass resolution pattern. The `getSlotsForType` method returns the best slots for a room, with profile-specific entries overriding generic ones:

```
public List<SlotEntry> getSlotsForType(String roomType, String
profile)                                                    {
                                                    ensureLoaded();
            List<SlotEntry> allSlots = slotsByType.getOrDefault(
                              roomType.toUpperCase(), List.of());
                        if (profile == null) {
                                    return allSlots.stream()
                                   .filter(s -> s.profile == null)
                                                    .toList();
                                                    }


    // Profile-aware: deduplicate by slot_name, profile-specific wins
       Map<String, SlotEntry> bestByName = new LinkedHashMap<>();
              // First pass: generic slots as baseline
                   for (SlotEntry slot : allSlots) {
                        if (slot.profile == null) {
                         bestByName.put(slot.slotName, slot);
                                                    }
                                                    }
            // Second pass: profile-specific slots override
                   for (SlotEntry slot : allSlots) {
                      if (profile.equals(slot.profile)) {
                         bestByName.put(slot.slotName, slot);
                                                    }
                                                    }
                   return new ArrayList<>(bestByName.values());
}
```

The `LinkedHashMap` preserves insertion order (generic first, then profile-specific overwrites), ensuring deterministic dispatch. The `SlotEntry` record captures room type, slot name, assembly ID, anchor face, priority, and profile — all loaded from the `ad_room_slot` table in the component library.

6.2 BeamTypeResolver: Span-Range Matching

The `BeamTypeResolver` resolves structural beam dimensions from metadata rules based on span length and construction profile:

```
public BeamTypeEntry resolve(String context, double spanM, String
profile)                                                        {
                                                        ensureLoaded();
                        if      (rules.isEmpty())       return      null;

                    //      Pass      1:      Profile-specific      rules
                            if       (profile       !=       null)       {
                            for    (BeamTypeRule    rule    :    rules)    {
                          if  (!rule.context.equals(context))  continue;
              if (rule.profile == null || !rule.profile.equals(profile))
continue;
                                  if   (matchesSpan(rule,   spanM))   {
                                return  beamTypes.get(rule.beamTypeId);
                                                                    }
                                                                    }
                                                                    }

                //   Pass   2:   Generic   rules   (profile   =   NULL)
                      for     (BeamTypeRule     rule     :     rules)     {
                          if   (!rule.context.equals(context))   continue;
                            if    (rule.profile    !=    null)    continue;
                                 if    (matchesSpan(rule,    spanM))    {
                                 return   beamTypes.get(rule.beamTypeId);
                                                                    }
                                                                    }


                                                    return            null;
}
```

Rules are loaded from `ad_beam_type_rule` sorted by priority (ascending). Each rule specifies a context (`FLOOR` or `LINTEL`), a span range `[span_min_m, span_max_m]`, a profile, and a target beam type. Returning `null` when no rule matches enables graceful degradation: residential profiles without beam rules skip frame generation entirely, while institutional profiles produce correctly-sized RC beams.

6.3 StoreyCompiler: 12-Step Pipeline

The `StoreyCompiler.compileStorey()` method implements the core compilation pipeline as a sequence of deterministic phases operating on a shared mutable context:

```
static StoreySpec compileStorey(StoreyDef storey, double baseZ,
                                boolean isGround, boolean isTop,
                                BuildingDefinition building,
                                SharedElementRegistry registry) {
    var ctx = new StoreyBuildContext(storey, baseZ, isGround, isTop,
                                     building, registry);
    resolveRoomLayout(ctx);
    compileCoreElements(ctx);
    resolveUnitInteriors(ctx);
    compileSlabAndPerimeter(ctx);
    compileInteriorWallsAndOpenings(ctx);
    placeMEPSprinklers(ctx);
    placeFixturesAndFurniture(ctx);
    placeStructural(ctx);
    placeHVAC(ctx);
    placeElectrical(ctx);
    placePlumbing(ctx);
    mepBomGapFill(ctx);
    return assembleStoreySpec(ctx);
}
```

The pipeline follows a strict ordering: spatial layout first (rooms, unit interiors), then enclosure (slab, perimeter walls, interior walls, openings), then content placement (MEP, furniture, structural), and finally a gap-fill pass for BOM-driven MEP elements. The `StoreyBuildContext` is a mutable carrier object that accumulates results across phases — each phase reads from and writes to the context. The final `assembleStoreySpec()` freezes the context into an immutable `StoreySpec` record.

6.4 BundleWorker: OSGi-Inspired Contract

The `BundleWorker` interface defines placement workers with explicit data contracts:

```
public interface BundleWorker {
    String themeId();
    String anchorFace();
    List<PlacedElement> execute(RoomEnvelope room, PlacementContext
ctx);
```

```
                                record          RoomEnvelope(
                        String    roomName,    String    roomType,
                    double   minX,   double   minY,   double   minZ,
                    double   maxX,   double   maxY,   double   maxZ,
                                    List<OpeningInfo>     openings,
                                  List<double[]>     reservedZones
                                                )              {
    public  double  width()    {  return  maxX  -  minX;  }
    public  double  depth()    {  return  maxY  -  minY;  }
     public  double  height()  {  return  maxZ  -  minZ;  }
                                                              }

                                record          PlacedElement(
                            String    name,    String    ifcClass,
                        double    x,    double    y,    double    z,
                double   width,   double   depth,   double   height,
                      double    rotation,    String    geometryHash,
                           String    role,    String    assemblyId
                                                )              {}
}
```

The `RoomEnvelope` provides workers with all spatial information needed for placement: room bounds, opening locations (for avoiding door swings), and zones reserved by prior workers (for avoiding collisions). The `PlacedElement` output includes a `geometryHash` linking to LOD 400 tessellated geometry in the component library, and a `role` within the assembly (e.g., `BED`, `NIGHTSTAND`, `WARDROBE`) for bill-of-materials traceability.

6.5 CatalogContract: Validation at Parse-Time

The `CatalogContract` enforces the DSL-as-catalog-selector principle:

```
public interface CatalogContract {
    record CatalogViolation(
        String referenceType, String referenceValue,
        String table, String message
    ) {}

    record CatalogResult(
        List<CatalogViolation> violations,
        int resolved, int total
    ) {
        public boolean isClean() { return violations.isEmpty(); }
        public double coveragePercent() {
            return total == 0 ? 100.0 : (resolved * 100.0 / total);
        }
    }

    CatalogResult validateCatalog();
}
```

The `CatalogValidator` implementation checks that every DSL reference — building template, unit type, floor BOM, room type, opening family, construction profile — resolves to an existing entry in the component library. Violations are collected with enough context (reference type, table name, message) for actionable error reporting. This catches the most common configuration errors: misspelled profile names, missing BOM entries, and references to deleted component families.

6.6 Spatial X-Ray Algorithm

The spatial checker's X-ray scoring algorithm (implemented in Python) compares element dimension signatures between compiled output and reference:

```
def spatial_xray(out_conn, ref_conn):
    CATEGORY_MAP = {
        "IfcPlate": "WALL", "IfcMember": "FRAME",
        "IfcWall": "WALL",
        "IfcBeam": "BEAM", "IfcColumn": "COLUMN",
        "IfcDoor": "DOOR", "IfcWindow": "WINDOW",
        "IfcFurniture": "FURNITURE",
        "IfcFurnishingElement": "FURNITURE",
        # ... additional mappings
    }

    def get_raw_elements(conn):
        rows = conn.execute("""
            SELECT m.ifc_class,
                r.maxX - r.minX AS dx,
                r.maxY - r.minY AS dy,
                r.maxZ - r.minZ AS dz
            FROM elements_meta m
            JOIN elements_rtree r ON m.id = r.id
        """).fetchall()

        elements = []
        for cls, dx, dy, dz in rows:
            cat = CATEGORY_MAP.get(cls, cls)
            dims = sorted([dx, dy, dz], reverse=True)
            exact_sig = (cat,
                round(dims[0] * 100) * 10,
                round(dims[1] * 100) * 10,
                round(dims[2] * 100) * 10)
            elements.append((cat, dims, exact_sig))
    return elements
```

The algorithm proceeds in three phases: (1) extract all element bounding boxes from both databases, (2) classify each element into a spatial category via the `CATEGORY_MAP`, and (3) compute near-matches using asymmetric tolerances — 5mm absolute for the thin dimension, 10% relative for the larger dimensions. The asymmetry reflects a domain insight: the thin dimension of walls, doors, and slabs is often dominated by mesh tessellation artifacts rather than design intent, while the length and height are meaningful spatial measures.

7. DSL Examples

7.1 SampleHouse: UK Residential (3 Rooms)

The SampleHouse DSL describes a single-storey UK house with three rooms — living room, entrance corridor, and bedroom:

```
BUILDING "Ifc4_SampleHouse" type:SINGLE_UNIT profile:"UK_Residential"
{

                                        GRID            {
                              axes:   A,   B,   C   /   1,   2,   3
                              spacing:   9.3,   4.5   /   2.0,   3.5
                                                        }

                 STOREY   "Ground   Floor"   level:0   height:3.3m   {
                                LIVING   "living"   bounds:A1-B3   {
                                           exterior:   west,   north;
                                                      WINDOW    north;
                                                      WINDOW    north;
                                                       WINDOW    north
                                                                 }
                                CORRIDOR   "entrance"   bounds:B1-C2   {
                                                     adjacent:   living;
                                                      exterior:    south;
                                           DOOR   south   type:D_EXT_DBL
                                                                 }
                                 BEDROOM   "bedroom"   bounds:B2-C3   {
                                                    adjacent:   entrance;
                                             exterior:   north,   east;
                                                         WINDOW    north
                                                                 }
                                                                 }

                               ROOF       pitch:0deg       overhang:600mm
}
```

Key DSL features visible in this example:

- **`profile:"UK_Residential"`** selects UK-specific wall types (290mm cavity brick), door families, and furniture BOM recipes from the metadata catalog.

- **`GRID`** defines a coordinate reference system. Axes A-C at spacings 9.3m and 4.5m produce X coordinates 0, 9.3, 13.8. Axes 1-3 at spacings 2.0m and 3.5m produce Y coordinates 0, 2.0, 5.5.
- **`bounds:A1-B3`** places the living room from grid intersection A1 to B3 — a 9.3m x 5.5m rectangle. No explicit dimensions; the grid determines size.
- **`exterior: west, north`** declares which faces are exterior walls. The compiler selects the appropriate exterior wall type for the UK_Residential profile.
- **`adjacent: living`** declares a room-to-room connection. The compiler places an interior door at the shared wall, selecting the appropriate interior door family.

The entire building description — 3 rooms, 4 windows, 1 door, adjacency relationships, grid layout — is 24 lines. The compiler resolves this into 108 positioned IFC elements: walls, openings, furniture, MEP terminals, structural members, finish slabs, and a roof.

7.2 SJTII Terminal: Malaysian Institutional (4 Storeys, 37 Rooms)

The Terminal DSL describes a 4-storey Malaysian airport terminal:

```
BUILDING    "SJTII_Terminal"    profile:"Malaysian_Institutional"    {

                                                    GRID             {
            axes:  A,  B,  C,  D,  E,  F,  G  /  1,  2,  3,  4,  5,  6,  7,  8
           spacing:  10,  10,  12,  12,  10,  8  /  8,  8,  8,  8,  8,  8,  8
                                                                     }

              STOREY    "Ground    Floor"    level:0    height:4.0m    {
                      LOBBY   "main_lobby"   bounds:A1-C3   {   ...   }
                     OPEN_PLAN   "checkin"   bounds:C1-E3   {   ...   }
                       OFFICE   "admin_1"   bounds:E1-F2   {   ...   }
                    CORRIDOR   "corridor_g"   bounds:A3-G4   {   ...   }
                  TOILET_BLOCK   "toilet_g1"   bounds:A4-B5   {   ...   }
                       DINING   "canteen"   bounds:C4-E6   {   ...   }
                STAIR  "stair_g"  at:F3  width:1.5m  to:"First  Floor"
                                                                    ...
                                                                     }

           STOREY   "First   Floor"   level:1   height:4.0m   {   ...   }
          STOREY   "Second   Floor"   level:2   height:4.0m   {   ...   }
           STOREY   "Third   Floor"   level:3   height:4.0m   {   ...   }

                               ROOF       pitch:0deg       overhang:600mm
}
```

The profile `"Malaysian_Institutional"` selects 150mm brick-plaster walls (93% of reference walls), RC beams sized by the span-range resolver (300x600mm to 300x750mm), canteen furniture sets (tables with 4 dining chairs each), and institutional opening families. The 7x8 grid (62m x 56m footprint) with variable spacing (10-12m X, 8m Y) was derived from column grid analysis of the reference building.

This DSL compiles to 2,726 elements across 22 IFC classes. The same compiler, the same resolver code, and the same compilation pipeline handle both the 3-room UK house and the 37-room Malaysian terminal — the difference is entirely in the metadata catalog.

---

8. Empirical Results

8.1 Score Progression

The following table shows ARC-discipline X-ray scores across development phases:

Phase	SampleHouse	Duplex	Terminal	Key Changes
118C	2%	5%	—	Initial Rosetta pairs established
119A	10%	5%	—	Wall thickness alignment
119D	17%	27%	—	Frame depth + opening depth fix
120	26%	37%	8%	Thesaurus alignment + Terminal 3rd stone
121	28%	40%	8%	Finish slabs + wall height + axis fix
122A	28%	40%	8%	STR grammar (beam overcount fixed)
122B	53%	42%	8%	Doors + multi-slot + BOM +

Phase	SampleHouse	Duplex	Terminal	Key Changes
				name matching
122C	62%	53%	8%	Kitchen cabinets + vanity + CENTER dining
122D	62%	53%	9%	Profile-aware columns + furniture + windows
Target	100%	100%	100%	All stones fully reproducible

Each score improvement corresponds to a specific compiler or metadata change, demonstrating the methodology's incrementality. The largest single-session gain was Phase 122B (+25% on SampleHouse from four targeted fixes: adjacency doors, multi-slot dispatch, BOM expansion, and name matching).

8.2 Per-Category Breakdown (Phase 122D)

Category	SampleHouse	Duplex	Terminal
Wall thickness	5/5 (100%)	48/57 (84%)	306/333 (91%)
Opening sizes	7/7 (100%)	16/38 (42%)	56/371 (15%)
Furniture sizes	10/14 (71%)	46/61 (75%)	25/176 (14%)
Slab/Roof	0/3 (0%)	2/21 (10%)	—
X-ray near	18/35 (51%)	84/183 (46%)	119/3,818 (3%)
ARC Overall	**62%**	**53%**	**9%**

Key observations:

- **Wall thickness** converges fastest across all traditions. Grammar Rule 1 (`WALL = SUM(layers)`) is universal: it produces correct thicknesses for UK cavity brick (290mm), US frame-with-siding (417mm), and Malaysian brick-plaster (150mm) from the same rule with different layer data.
- **Opening sizes** are well-matched for SampleHouse (100%) but lag for the Terminal (15%), where curtain wall windows (1310 x 3450mm) require opening families not yet in the catalog.
- **Furniture** shows strong results (71-75%) for residential buildings where BOM recipes are mature, but lower for institutional buildings where quantity scaling (60 canteen tables vs. 4 output) is needed.
- The **Terminal** at 9% represents the system's current limitation with large institutional buildings: most of the gap is due to missing opening families, quantity scaling, and position alignment rather than fundamental architectural mismatches.

8.3 Cross-Tradition Grammar Validation

The 14 grammar rules discovered through the Rosetta process span three construction traditions:

Rule	Description	UK	US	MY
1	WALL = SUM(layers)	290mm	417mm	150mm
2	OPENING.thin = frame_depth	199mm	467mm	150mm
3	FURNITURE = catalog(profile, room)	BED_SET	BED_SET	CANTEEN_SET
4	SLAB = structural + finish	170+19mm	200+19mm	200+13mm
5	OPENING.position = wall_midpoint	Verified	Verified	Verified
6	STAIR.width = code_minimum	900mm	900mm	1500mm
7	GRID → column_positions	9.3/4.5m	6/6m	8/10-12m
8	ROOF = perimeter + overhang	600mm	600mm	600mm
9	PIPE.dia < WALL.thick	—	Verified	Verified
10	CERAMIC_FINISH = wet_room	—	—	Verified

Rule	Description	UK	US	MY
11	COLUMN at WALL junctions (76%)	—	—	Verified
12	BEAM depth/span = 0.10-0.15	—	—	Verified
13	CEILING_VOID ~ 900-1000mm	—	—	Verified
14	SPRINKLER_DROP = 500-1200mm	—	—	Verified

Rules 1-8 are verified across residential buildings (UK + US). Rules 9-14 are discovered from the multi-discipline institutional building (Malaysian Terminal). The distinction between “verified” and “—” reflects which stones demonstrate each rule, not applicability limits. For example, Rule 9 (pipes fit inside walls) likely holds for UK residential but has not been measured because the SampleHouse reference contains no MEP elements.

---

9. Challenges and Lessons Learned

9.1 The Thin-Dimension Problem

The most persistent challenge in spatial comparison is the thin dimension of building elements. A reference IFC door has a bounding box depth of 199mm (frame thickness). The compiler's library stores a tessellated door mesh with a depth of 150mm (panel thickness). Both represent the same door, but the spatial X-ray sees them as different elements.

The solution was Grammar Rule 2: `OPENING.thin = frame_depth`. The `ad_opening_family` table now stores a `depth_mm` value that overrides the mesh bounding box when writing the element's spatial envelope. The mesh geometry is preserved for LOD 400 visualization; only the bounding box is corrected for spatial comparison.

This required a conceptual shift: the X-ray should compare *spatial bone structure* (where walls start and end, where doors are placed), not *mesh soft tissue* (exact vertex positions). We adopted asymmetric tolerances — tight (5mm) for the thin dimension, loose (10%) for the larger dimensions — to reflect this distinction.

9.2 Discipline Mapping Conflicts

IFC classes can belong to different disciplines depending on context. `IfcColumn` is classified as structural (STR) in the IFC schema, but in Rosetta Stone comparison, reference columns appear in the architectural (ARC) discipline because they were drawn by architects, not structural engineers.

The fix required modifying `inferDiscipline()` to check element GUID prefixes *before* consulting the type-to-discipline mapping table. Elements with `COLUMN_` GUID prefixes are tagged ARC; everything else follows the standard mapping. This was a two-line fix in Java but required a corresponding change in the Python spatial checker's hardcoded `DISCIPLINE_FOR_CLASS` dictionary.

Lesson: Any discipline classification must be kept in sync across all tools in the pipeline. A classification change in one place without updating the other creates score artifacts that are hard to diagnose.

9.3 Profile Resolution Ordering

When both generic and profile-specific metadata exist for the same concept (e.g., a `FURNITURE` slot for `BEDROOM` exists both with `profile=NULL` and `profile=Malaysian_Institutional`), the resolution order matters. Early implementations used simple list iteration, which could return generic results even when profile-specific data was available.

The fix was the two-pass pattern described in Section 5.2: all resolvers now perform a profile-specific pass first, then a generic fallback pass. The `LinkedHashMap` deduplication in `SlotRegistry` ensures that profile-specific entries overwrite generic entries for the same slot name without duplicating entries that have no profile-specific override.

9.4 Geometry Coordinate Conventions

The component library stores mesh geometry with a specific axis convention: `widthMm` is the Y-extent and `depthMm` is the X-extent. This is counterintuitive (depth is typically the Y-axis in screen coordinates) and caused placement errors when the convention was assumed to be the opposite.

Additionally, the `elements_rtree` table uses column order `(id, minX, maxX, minY, maxY, minZ, maxZ)` — interleaving min/max per axis — while the `BoundingBox` Java record uses `(minX, minY, minZ, maxX, maxY, maxZ)` — grouping all mins, then all maxes. This impedance mismatch caused at least two bounding-box corruption bugs before being documented as a standing trap.

9.5 BOM Completeness and Silent Failures

SQLite's `INSERT OR IGNORE` silently drops rows that violate NOT NULL constraints. An `ad_bom` entry with a missing `bom_name` field was silently ignored, causing the BOM tree to fail to load for that assembly. No error was reported; the room simply had no furniture.

Lesson: Always use `INSERT OR REPLACE` with all required columns explicitly provided, and log BOM tree depth during loading to catch silent drops. A furniture-less room should be an anomaly, not a silent default.

9.6 Blended Scores Mislead

The Duplex has 890 MEP elements (82% of total) but only 3% MEP match rate. A blended score of 13% obscures the fact that architectural fidelity is 37%. Conversely, improving MEP by a few percentage points would dramatically improve the blended score without any architectural progress.

Lesson: Always track per-discipline scores. Use the ARC-discipline score as the primary convergence metric. Blended scores are meaningful only when all disciplines are above a useful threshold.

---

10. Road Ahead

10.1 100% Convergence

The immediate goal is 100% spatial X-ray fidelity on all three Rosetta stones. The remaining gaps are well-characterized:

- **SampleHouse (62% -> 100%)**: Profile-specific furniture (UK bed, UK coffee table), armchair library component, remaining dining chair placement, curtain wall panels.
- **Duplex (53% -> 100%)**: Wall dimension exact-matching, slab dimension alignment, remaining furniture gaps, railing generation.
- **Terminal (9% -> 100%)**: Window and door family expansion (curtain wall and institutional sizes), quantity scaling for repetitive furniture, slab generation per-room, column XY position alignment.

10.2 Infrastructure Transfer

The grammar rules discovered from building stones partially transfer to infrastructure (bridges, roads, rail). The methodology always transfers. We have identified four transferable idioms:

Building Idiom	Abstract Idiom	Infrastructure Example
Room	ZONE	Lane, Span, Section
Grid	REFERENCE_SYSTEM	Alignment, Chainage
Storey	DECOMPOSITION	Station, Span
Wall	BOUNDARY	Barrier, Retaining wall

The universal grammar sentence — "ZONE has CONTENT placed via REFERENCE_SYSTEM, decomposed by DECOMPOSITION, bounded by BOUNDARY, penetrated by PENETRATION" — describes both a bedroom with a bed and a bridge span with a bearing pad. The slot dispatch pattern (`ad_room_slot` -> `SlotRegistry` -> worker dispatch) does not care what the zone is; it cares that the zone has a type and the type has slots.

10.3 Multi-Discipline Compilation

The Terminal stone demonstrates that engineering disciplines are not independent. Architectural decisions constrain MEP routing (pipes fit inside walls — 99.3% of the time). Structural grids quantize architectural layout (76% of columns at wall junctions). Wall finish encodes room function (ceramic = wet room). The grammar must encode these cross-discipline constraints rather than treating each discipline as a separate compilation pass.

10.4 Building Template Expansion

The current DSL still specifies individual rooms. The catalog-selector principle demands that a building type should be fully expandable from a single line:

```
BUILDING "My Terminal" type:TERMINAL_4F profile:"Malaysian_Institutional"
```

This requires `ad_building_template` and `ad_floor_template` tables that encode room counts, grid positions, and assembly selections per building type. The metadata already has 9 building templates and 12 floor types; the remaining work is parser support for type-based expansion and template-to-room resolution.

10.5 Additional Rosetta Stones

Each new stone makes the next one cheaper — like language acquisition. The first language is hardest, the second shares cognates, the third is faster still. Candidate stones include:

- **European residential** (German, French standards) — tests Rule 1 against insulated cavity walls.
- **Healthcare facility** — tests clean room MEP constraints and specialized room types.
- **High-rise commercial** — tests vertical transportation (elevators, shafts) and curtain wall systems.
- **Infrastructure (bridge)** — tests the ZONE/REFERENCE_SYSTEM/DECOMPOSITION abstraction.

---

11. Conclusion

We have presented the BIM Intent Compiler, a system that compiles declarative building descriptions into spatially-verified IFC output. The three-tier architecture (DSL / Metadata / Resolver) separates user intent from construction knowledge from compilation logic, enabling progressive enrichment of the metadata catalog without code changes. The Rosetta Stone Methodology provides a rigorous, reproducible validation framework that measures compiler fidelity against real buildings as a single comparable number.

The system's current state — 62% fidelity on a UK house, 53% on a US duplex, 9% on a Malaysian terminal — demonstrates that declarative BIM compilation is feasible across construction traditions. The 14 grammar rules discovered from first principles show that construction knowledge can be formalized as mathematical relations applicable across building types and regional standards. The path to 100% convergence is well-characterized and primarily requires catalog expansion rather than architectural changes.

The deeper contribution is methodological. The Linguist's Method (Dictionary -> Thesaurus -> Grammar) provides a systematic approach to discovering and validating the rules that govern building composition — rules that were previously implicit in practitioners' expertise. By encoding these rules in queryable metadata and proving them against reference buildings, the system captures construction knowledge in a form that is transparent, version-controlled, and machine-executable. This represents a step toward treating building design as a formal language with a provable grammar, rather than an art practiced through graphical manipulation.

References

[1] C. Eastman, P. Teicholz, R. Sacks, and K. Liston, *BIM Handbook: A Guide to Building Information Modeling for Owners, Managers, Designers, Engineers, and Contractors*. Hoboken, NJ: John Wiley & Sons, 2008.

[2] R. Sacks, C. Eastman, G. Lee, and P. Teicholz, *BIM Handbook: A Guide to Building Information Modeling for Owners, Managers, Designers, Engineers, and Contractors*, 3rd ed. Hoboken, NJ: John Wiley & Sons, 2018.

[3] International Organization for Standardization, "ISO 16739-1:2018 — Industry Foundation Classes (IFC) for data sharing in the construction and facilities management industries — Part 1: Data schema," 2018.

[4] buildingSMART International, "IFC4 Add2 TC1 Documentation," 2017. [Online]. Available: https://standards.buildingsmart.org/IFC/RELEASE/IFC4/ADD2_TC1/HTML/

[5] M. Venugopal, C. M. Eastman, R. Sacks, and J. Teizer, "Semantics of model views for information exchanges using the industry foundation class schema," *Advanced Engineering Informatics*, vol. 26, no. 2, pp. 411-428, 2012.

[6] C. Preidel, A. Borrmann, and Y. A. Elzonbli, "BIM-based code compliance checking," in *Building Information Modeling: Technology Foundations and Industry Practice*, A. Borrmann, M. König, C. Koch, and J. Beetz, Eds. Cham: Springer, 2018, pp. 367-381.

[7] J. Won and G. Lee, "How to tell if a BIM project is successful: A goal-driven approach," *Automation in Construction*, vol. 69, pp. 34-43, 2016.

[8] P. Pauwels, S. Zhang, and Y.-C. Lee, "Semantic web technologies in AEC industry: A literature overview," *Automation in Construction*, vol. 73, pp. 145-165, 2017.

[9] H. Lee, S. Lee, S. Park, and J. Kim, "Translating building legislation into a computer-executable format for evaluating building permit requirements," *Automation in Construction*, vol. 71, pp. 49-61, 2016.

[10] W. Solihin and C. Eastman, "Classification of rules for automated BIM rule checking development," *Automation in Construction*, vol. 53, pp. 69-82, 2015.

[11] M. Fowler, *Domain-Specific Languages*. Upper Saddle River, NJ: Addison-Wesley, 2010.

[12] M. Mernik, J. Heering, and A. M. Sloane, “When and how to develop domain-specific languages,” *ACM Computing Surveys*, vol. 37, no. 4, pp. 316-344, 2005.

[13] A. van Deursen, P. Klint, and J. Visser, “Domain-specific languages: An annotated bibliography,” *ACM SIGPLAN Notices*, vol. 35, no. 6, pp. 26-36, 2000.

[14] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Pearson/Addison-Wesley, 2006.

[15] T. Parr, *Language Implementation Patterns: Create Your Own Domain-Specific and General Programming Languages*. Raleigh, NC: Pragmatic Bookshelf, 2009.

[16] OSGi Alliance, “OSGi Core Release 8 Specification,” 2020. [Online]. Available: https://docs.osgi.org/specification/

[17] A. Robinson, *The Last Man Who Knew Everything: Thomas Young, the Anonymous Polymath Who Proved Newton Wrong, Explained How We See, Cured the Sick, and Deciphered the Rosetta Stone*. New York: Pi Press, 2006.

[18] J. T. Champollion, *Lettre à M. Dacier relative à l’alphabet des hiéroglyphes phonétiques*. Paris: Firmin Didot, 1822.

[19] A. Robinson, *Lost Languages: The Enigma of the World’s Undeciphered Scripts*. New York: McGraw-Hill, 2002.

[20] BIMForum, “Level of Development (LOD) Specification Part I & Commentary,” 2023. [Online]. Available: https://bimforum.org/lod/

[21] T. Liebich, “IFC4 — the new buildingSMART standard,” in *Proc. International Conference on Computing in Civil and Building Engineering*, 2013.

[22] J. Steel, R. Drogemuller, and B. Toth, “Model interoperability in building information modelling,” *Software & Systems Modeling*, vol. 11, no. 1, pp. 99-109, 2012.

[23] J. Zhang and N. M. El-Gohary, “Automated information transformation for automated regulatory compliance checking in construction,” *Journal of Computing in Civil Engineering*, vol. 29, no. 4, 2015.

[24] A. Borrmann, M. König, C. Koch, and J. Beetz, Eds., *Building Information Modeling: Technology Foundations and Industry Practice*. Cham: Springer, 2018.

[25] G. Lee, R. Sacks, and C. M. Eastman, "Specifying parametric building object behavior (BOB) for a building information modeling system," *Automation in Construction*, vol. 15, no. 6, pp. 758-776, 2006.

[26] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.

[27] SQLite Consortium, "The SQLite R*Tree Module," 2024. [Online]. Available: https://www.sqlite.org/rtree.html

[28] S. Lockley, C. Benghi, and M. Cerny, "Xbim: An open-source toolkit for IFC," *Journal of Open Research Software*, vol. 5, no. 1, 2017.

[29] J. Beetz, J. van Leeuwen, and B. de Vries, "IfcOWL: A case of transforming EXPRESS schemas into OWL ontologies," in *Proc. Artificial Intelligence in Engineering Design, Analysis and Manufacturing*, vol. 23, no. 1, pp. 89-101, 2009.

[30] M. H. Rasmussen, P. Pauwels, M. Lefrançois, and G. F. Schneider, "Building topology ontology," W3C, 2019. [Online]. Available: https://w3c-lbd-cg.github.io/bot/

